МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 10

Алгоритмы поиска кратчайших путей и расчёта критического пути сетевого графика

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить алгоритмы оптимизации на взвешенных графах, реализовать алгоритм Дейкстры в двух вариантах и применить топологическую сортировку к расчёту календарного плана проекта методом критического пути.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Графы для Дейкстры	разреженный и плотный
Размеры n	300; 600; 1200
Веса дуг	1…80
Число работ в проекте	13
Дополнительное задание	К7 — диаграмма Ганта с выделением работ критического пути

Уточнения, принятые при выполнении. Для варианта 14 параметры «разреженный» и «плотный» граф не заданы числом; приняты значения варианта 0: разреженный граф — $m \approx 4n$ дуг, плотный — 20 % от максимально возможного числа дуг $n(n-1)$. Графы ориентированные, веса — случайные целые, каждая вершина достижима из вершины 0 (граф содержит остов из дуг $j \to i$, $j < i$). Проект из 13 работ получен из базового набора (10 работ) добавлением трёх работ K, L, M с сохранением ацикличности и параллельных ветвей.

2. Алгоритмы и оценки трудоёмкости

2.1. Алгоритм Дейкстры

Для вершины-источника s поддерживаются оценки $d[v]$ (изначально $d[s] = 0$, остальные ∞). На каждом шаге выбирается ещё не просмотренная вершина v с наименьшей оценкой, и выполняется релаксация исходящих дуг: если $d[v]+w(v,u) < d[u]$, то $d[u] := d[v]+w(v,u)$, $parent[u] := v$. Корректность опирается на неотрицательность весов (у нас веса от 1 до 80) и на оптимальную подструктуру: подпуть кратчайшего пути — кратчайший.

Таблица 2. Две реализации

Реализация	Поиск минимума	Трудоёмкость
Линейный поиск по массиву	$\Theta(n)$ за шаг	$\Theta(n^2 + m)$, то есть $\Theta(n^2)$
Приоритетная очередь (`heapq`)	$O(\log n)$ за операцию	$O(m \log n)$

Для разреженного графа ($m = 4n$) куча даёт $O(n \log n)$ против $\Theta(n^2)$ у массива; для плотного ($m = \Theta(n^2)$) оценки сближаются ($O(n^2 \log n)$ против $\Theta(n^2)$), и выигрыш определяется константами. Реализация с кучей использует «ленивое удаление»: устаревшие записи пропускаются при извлечении. Путь восстанавливается по массиву `parent` за $O(\text{длина пути})$.

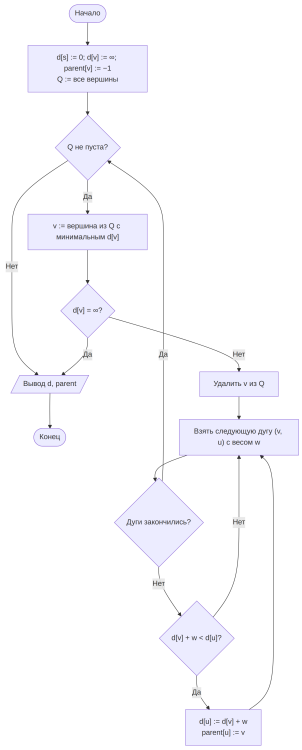
2.2. Топологическая сортировка и критический путь

Сетевой график проекта — ациклический ориентированный граф: вершины — работы, дуги — предшествование. Топологическая сортировка (алгоритм Кана, $\Theta(n+m)$) упорядочивает работы так, что предшественники идут раньше; если упорядочены не все работы, график содержит цикл (ошибка исходных данных).

Прямой проход: $ES_i = \max_{j \in \mathrm{Pred}(i)} EF_j$, $EF_i = ES_i + t_i$ (для работ без предшественников $ES_i = 0$), срок проекта $T = \max_i EF_i$. Обратный проход: $LF_i = \min_{j \in \mathrm{Succ}(i)} LS_j$, $LS_i = LF_i - t_i$ (для работ без последователей $LF_i = T$). Полный резерв: $R_i = LS_i - ES_i$. Критические работы имеют $R_i = 0$. Трудоёмкость $\Theta(n + m)$.

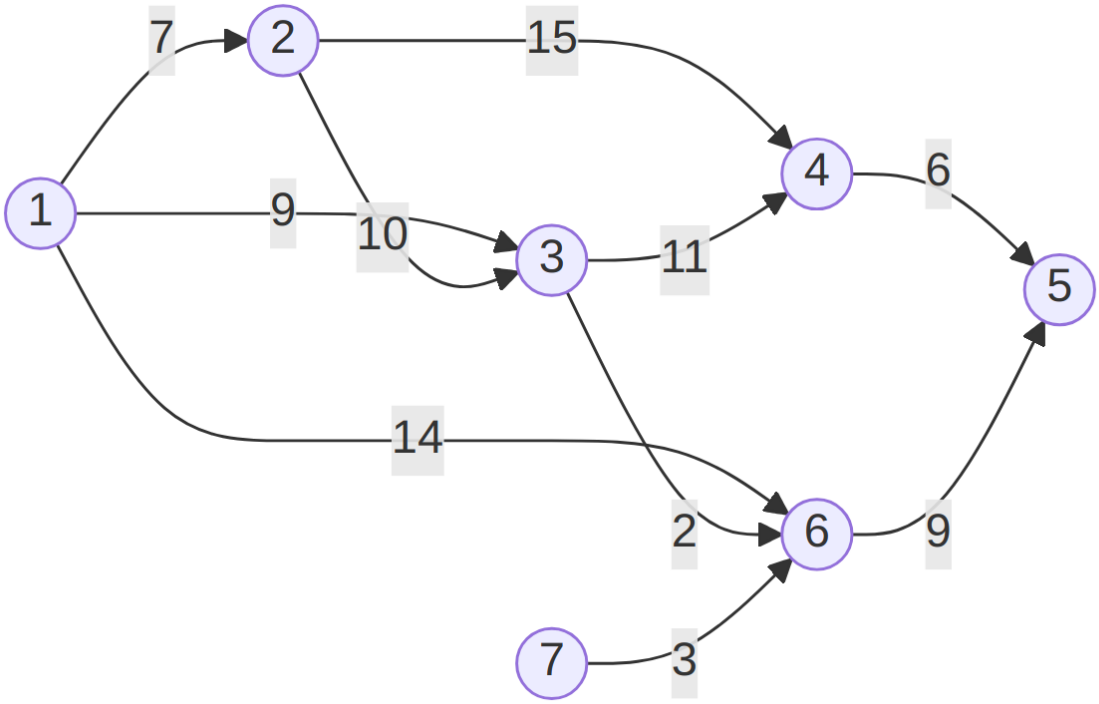
3. Тестовый граф и ручной расчёт кратчайших путей

3.1. Блок-схема алгоритма Дейкстры



3.2. Тестовый граф

Ориентированный взвешенный граф из 7 вершин (веса на дугах); источник — вершина 1. Вершина 7 имеет только исходящую дугу и недостижима из вершины 1.



3.3. Ручной расчёт и результат программы

Расчёт по шагам алгоритма (порядок выбора вершин определяется минимальной оценкой):

1. Источник: $d[1] = 0$, остальные ∞. Выбираем вершину 1; релаксация дуг: $d[2] = 7$, $d[3] = 9$, $d[6] = 14$.
2. Минимум среди непросмотренных — вершина 2 ($d = 7$). Дуга $2 \to 3$: $7 + 10 = 17 > 9$ — не улучшает; дуга $2 \to 4$: $d[4] = 7 + 15 = 22$.
3. Вершина 3 ($d = 9$). Дуга $3 \to 4$: $9 + 11 = 20 < 22$, $d[4] = 20$; дуга $3 \to 6$: $9 + 2 = 11 < 14$, $d[6] = 11$.
4. Вершина 6 ($d = 11$). Дуга $6 \to 5$: $d[5] = 11 + 9 = 20$.
5. Вершины 4 и 5 ($d = 20$; шаги 5 и 6 протокола): релаксация дуги $4 \to 5$ даёт $20 + 6 = 26 > 20$ — без изменений.
6. Вершина 7 остаётся с $d = \infty$ (недостижима).

Таблица 3. Результат программы (протокол шагов), совпадает с ручным расчётом

Шаг	Выбрана вершина v	$d[v]$	Релаксация: обновлённые оценки	Оценки $d[1..7]$ после шага
1	1	0	$d[2] = 7$; $d[3] = 9$; $d[6] = 14$	(0, 7, 9, ∞, ∞, 14, ∞)
2	2	7	$d[4] = 22$	(0, 7, 9, 22, ∞, 14, ∞)
3	3	9	$d[4] = 20$; $d[6] = 11$	(0, 7, 9, 20, ∞, 11, ∞)
4	6	11	$d[5] = 20$	(0, 7, 9, 20, 20, 11, ∞)
5	4	20	—	(0, 7, 9, 20, 20, 11, ∞)
6	5	20	—	(0, 7, 9, 20, 20, 11, ∞)

Итог: $d = (0, 7, 9, 20, 20, 11, \infty)$. Кратчайший путь до вершины 5: $1 \to 3 \to 6 \to 5$ (длина $9 + 2 + 9 = 20$); до вершины 4: $1 \to 3 \to 4$ (длина $9 + 11 = 20$). Отметим: путь до вершины 4 через вершину 2 длиннее ($7 + 15 = 22$) — это показывает смысл релаксации. Обе реализации Дейкстры, а также Беллмана — Форд (эталон) на 200 случайных графах дали одинаковые расстояния; вес каждого восстановленного пути равен расстоянию.

4. Сетевой график проекта

Таблица 4. Работы проекта (13 работ; базовые A–J и добавленные K, L, M)

Код	Наименование работы	Длительность, дн.	Предшественники
A	Сбор требований	5	—
B	Анализ бизнес-процессов	7	A
C	Проектирование базы данных	4	B
D	Проектирование интерфейса	6	B
E	Разработка серверной части	10	C
F	Разработка клиентской части	9	D
G	Интеграция модулей	3	E, F
H	Подготовка документации	4	C, D
I	Тестирование	6	G
J	Обучение пользователей	2	H, I
K	Развёртывание инфраструктуры	3	B
L	Миграция данных	5	C, K
M	Приёмка заказчиком	3	J, L

Сетевой график (в вершинах — код работы и длительность в днях; красным выделены критические работы, определённые в разделе 8):

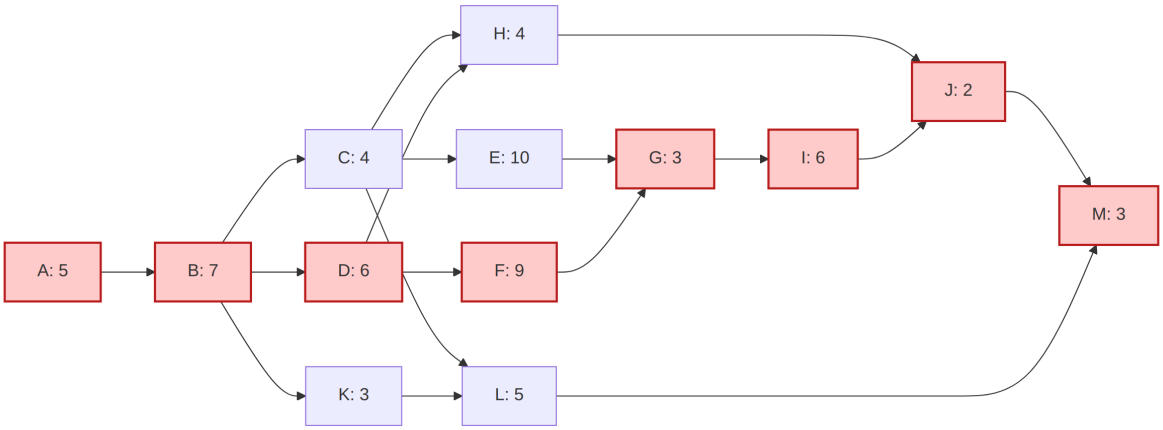


График ацикличен и содержит параллельные ветви (C–E и D–F; K и L), как требует задание. Контроль корректности исходных данных: если в график искусственно ввести цикл (сделать работу I предшественником B), топологическая сортировка завершается ошибкой, и программа выдаёт сообщение «Сетевой график содержит цикл».

5. Листинг программы

Программа содержит обе реализации Дейкстры, протокол шагов для малого графа, эталон Беллмана — Форда для проверки, генератор графов, расчёт критического пути (топологическая сортировка и два прохода), проверку полным перебором путей, замеры времени и построение графиков (включая диаграмму Ганта).

```
"""
Лабораторная работа № 10. Вариант 14.
Алгоритм Дейкстры (две реализации), восстановление пути и расчёт критического пути
проекта из 13 работ. Графы: разреженный (m ≈ 4n) и плотный (20 % дуг),
n = 300, 600, 1200, веса дуг 1…80. Дополнительное задание К7: диаграмма Ганта.
"""
import gc
import heapq
import json
import random
import statistics
import time
from collections import deque

INF = float("inf")
MAX_W = 80


# ---------------------------------------------------- алгоритм Дейкстры
def dijkstra_array(adj, s):
    """Дейкстра с линейным поиском минимума. Трудоёмкость θ(n²)."""
    n = len(adj)
    dist, parent, used = [INF] * n, [-1] * n, [False] * n
    dist[s] = 0
    for _ in range(n):
        v, best = -1, INF
        for i in range(n):                          # линейный поиск минимума
            if not used[i] and dist[i] < best:
                v, best = i, dist[i]
        if v == -1:
            break
        used[v] = True
        for u, w in adj[v]:                         # релаксация исходящих дуг
            if dist[v] + w < dist[u]:
```

```
                dist[u] = dist[v] + w
                parent[u] = v
    return dist, parent


def dijkstra_heap(adj, s):
    """Дейкстра с приоритетной очередью (heapq). Трудоёмкость O(m log n)."""
    n = len(adj)
    dist, parent = [INF] * n, [-1] * n
    dist[s] = 0
    heap = [(0, s)]
    while heap:
        d, v = heapq.heappop(heap)
        if d > dist[v]:                          # устаревшая запись
            continue
        for u, w in adj[v]:
            if d + w < dist[u]:
                dist[u] = d + w
                parent[u] = v
                heapq.heappush(heap, (dist[u], u))
    return dist, parent


def dijkstra_trace(adj, s):
    """Дейкстра с протоколом шагов (для ручной проверки на малом графе)."""
    n = len(adj)
    dist, parent, used = [INF] * n, [-1] * n, [False] * n
    dist[s] = 0
    steps = []
    for _ in range(n):
        v, best = -1, INF
        for i in range(n):
            if not used[i] and dist[i] < best:
                v, best = i, dist[i]
        if v == -1:
            break
        used[v] = True
        updates = []
        for u, w in adj[v]:
            if dist[v] + w < dist[u]:
                dist[u] = dist[v] + w
                parent[u] = v
                updates.append((u, dist[u]))
        steps.append((v, best, updates, list(dist)))
    return steps, dist, parent


def restore_path(parent, target):
    """Восстановление пути по массиву предков."""
    path = []
    while target != -1:
        path.append(target)
        target = parent[target]
    return path[::-1]


def bellman_ford(adj, s):
    """Эталон для проверки (Беллман — Форд, Θ(nm))."""
    n = len(adj)
    dist = [INF] * n
```

```
    dist[s] = 0
    for _ in range(n - 1):
        changed = False
        for v in range(n):
            if dist[v] < INF:
                for u, w in adj[v]:
                    if dist[v] + w < dist[u]:
                        dist[u] = dist[v] + w
                        changed = True
        if not changed:
            break
    return dist


def generate_graph(n, m, rng, max_weight=MAX_W):
    """Случайный ориентированный взвешенный граф: n вершин, m различных дуг.
    Сначала строится остов (каждая вершина достижима из вершины 0), затем
    добавляются случайные дуги до m."""
    arcs = set()
    for i in range(1, n):
        arcs.add((rng.randrange(i), i))
    while len(arcs) < m:
        u, v = rng.randrange(n), rng.randrange(n)
        if u != v:
            arcs.add((u, v))
    adj = [[] for _ in range(n)]
    for u, v in sorted(arcs):
        adj[u].append((v, rng.randint(1, max_weight)))
    return adj


# ---------------------------------------------------------- критический путь
def topological_order(tasks):
    """Топологическая сортировка (алгоритм Кана). tasks: код → (длительность, предшественники).
    При наличии цикла (некорректный сетевой график) выбрасывает ValueError."""
    indeg = {k: len(v[1]) for k, v in tasks.items()}
    succ = {k: [] for k in tasks}
    for k, (_, preds) in tasks.items():
        for p in preds:
            succ[p].append(k)
    queue = deque(sorted(k for k, d in indeg.items() if d == 0))
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for u in succ[v]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    if len(order) != len(tasks):
        raise ValueError("Сетевой график содержит цикл")
    return order, succ


def critical_path(tasks):
    """Расчёт ES, EF, LS, LF, резервов и критических работ."""
    order, succ = topological_order(tasks)
    ES, EF = {}, {}
    for v in order:                              # прямой проход
        duration, preds = tasks[v]
```

```
        ES[v] = max((EF[p] for p in preds), default=0)
        EF[v] = ES[v] + duration
    T = max(EF.values())
    LS, LF = {}, {}
    for v in reversed(order):                    # обратный проход
        duration = tasks[v][0]
        LF[v] = min((LS[u] for u in succ[v]), default=T)
        LS[v] = LF[v] - duration
    reserve = {v: LS[v] - ES[v] for v in tasks}
    critical = [v for v in order if reserve[v] == 0]
    return T, ES, EF, LS, LF, reserve, critical, order


def all_paths_longest(tasks):
    """Проверка полным перебором: все пути от начальных работ до конечных,
    возвращает (длина самого длинного пути, список самых длинных путей)."""
    succ = {k: [] for k in tasks}
    for k, (_, preds) in tasks.items():
        for p in preds:
            succ[p].append(k)
    best, best_paths = -1, []

    def walk(v, path, length):
        nonlocal best, best_paths
        path = path + [v]
        length += tasks[v][0]
        if not succ[v]:
            if length > best:
                best, best_paths = length, [path]
            elif length == best:
                best_paths.append(path)
            return
        for u in succ[v]:
            walk(u, path, length)

    for k, (_, preds) in tasks.items():
        if not preds:
            walk(k, [], 0)
    return best, best_paths


# Сетевой график проекта (вариант 14: 13 работ; базовая таблица + работы K, L, M)
NAMES = {"A": "Сбор требований", "B": "Анализ бизнес-процессов", "C": "Проектирование базы данных",
         "D": "Проектирование интерфейса", "E": "Разработка серверной части",
         "F": "Разработка клиентской части", "G": "Интеграция модулей",
         "H": "Подготовка документации", "I": "Тестирование", "J": "Обучение пользователей",
         "K": "Развёртывание инфраструктуры", "L": "Миграция данных", "M": "Приёмка заказчиком"}
TASKS = {"A": (5, []), "B": (7, ["A"]), "C": (4, ["B"]), "D": (6, ["B"]), "E": (10, ["C"]),
         "F": (9, ["D"]), "G": (3, ["E", "F"]), "H": (4, ["C", "D"]), "I": (6, ["G"]),
         "J": (2, ["H", "I"]), "K": (3, ["B"]), "L": (5, ["C", "K"]), "M": (3, ["J", "L"])}


# ---------------------------------------------------------------- проверки
def self_test():
    """Сверка обеих реализаций Дейкстры с Беллманом — Фордом, проверка путей."""
    rng = random.Random(10)
    for _ in range(200):
        n = rng.randint(5, 60)
        m = rng.randint(n - 1, min(n * (n - 1), 5 * n))
        adj = generate_graph(n, m, rng)
```

```
        d1, p1 = dijkstra_array(adj, 0)
        d2, p2 = dijkstra_heap(adj, 0)
        d3 = bellman_ford(adj, 0)
        assert d1 == d2 == d3
        weights = {(v, u): w for v in range(n) for u, w in adj[v]}
        for t in range(n):                          # вес восстановленного пути равен расстоянию
            path = restore_path(p2, t)
            assert path[0] == 0 and sum(weights[(path[i], path[i + 1])] for i in range(len(path) -
            ↪  1)) == d2[t]
    return 200


def timed(func, *args, repeats=5):
    times = []
    for _ in range(repeats):
        gc.collect()
        t0 = time.perf_counter()
        func(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times)


# ---------------------------------------------------------------- графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    for ax, kind, title in ((axes[0], "sparse", "Разреженный граф (m ≈ 4n)"),
                            (axes[1], "dense", "Плотный граф (20 % дуг)")):
        rows = [r for r in res["dijkstra"] if r["kind"] == kind]
        ns = [r["n"] for r in rows]
        ax.loglog(ns, [r["t_array"] for r in rows], "o-", label="линейный поиск минимума")
        ax.loglog(ns, [r["t_heap"] for r in rows], "s-", label="приоритетная очередь")
        ax.set_xlabel("число вершин n"); ax.set_ylabel("время, с"); ax.set_title(title)
        ax.grid(True, which="both", alpha=.3); ax.legend()
    plt.tight_layout(); plt.savefig("img/lr10_dijkstra.png", dpi=170); plt.close()

    cp = res["cpm"]
    order = sorted(cp["ES"], key=lambda k: (cp["ES"][k], k))
    plt.figure(figsize=(9, 5))
    for i, k in enumerate(order):
        crit = cp["reserve"][k] == 0
        plt.barh(i, cp["EF"][k] - cp["ES"][k], left=cp["ES"][k], color="#d62728" if crit else
↪   "#1f77b4",
                 edgecolor="k")
        if not crit:                                # полный резерв времени — светлая полоса
            plt.barh(i, cp["reserve"][k], left=cp["EF"][k], color="#c7dcef", edgecolor="#1f77b4",
↪   hatch="//")
        plt.text(cp["ES"][k] + 0.3, i, k, va="center", color="white", fontweight="bold")
    plt.yticks(range(len(order)), [f"{k}: {NAMES[k]}" for k in order], fontsize=8)
    plt.gca().invert_yaxis()
    plt.axvline(cp["T"], color="gray", ls="--")
    from matplotlib.patches import Patch
    plt.legend(handles=[Patch(color="#d62728", label="критические работы"),
                        Patch(color="#1f77b4", label="некритические работы"),
                        Patch(facecolor="#c7dcef", edgecolor="#1f77b4", hatch="//", label="полный
↪   резерв")],
               loc="upper right", fontsize=8)
```

```
    plt.xlabel("день проекта"); plt.title(f"Диаграмма Ганта проекта (T = {cp['T']} дн.)")
    plt.grid(True, axis="x", alpha=.3); plt.tight_layout(); plt.savefig("img/lr10_gantt.png",
↪   dpi=170); plt.close()


# ---------------------------------------------------------------------- main
if __name__ == "__main__":
    res = {"selftest": self_test()}
    print("Самопроверка: графов", res["selftest"], "— Дейкстра (оба варианта) = Беллман — Форд")

    # --- 1. Малый тестовый граф (вершины нумеруются с 1)
    small_arcs = [(1, 2, 7), (1, 3, 9), (1, 6, 14), (2, 3, 10), (2, 4, 15), (3, 4, 11),
                  (3, 6, 2), (4, 5, 6), (6, 5, 9), (7, 6, 3)]
    adj = [[] for _ in range(7)]
    for u, v, w in small_arcs:
        adj[u - 1].append((v - 1, w))
    steps, dist, parent = dijkstra_trace(adj, 0)
    res["small"] = {"arcs": small_arcs,
                    "steps": [{"v": v + 1, "d": d, "upd": [(u + 1, x) for u, x in upd],
                               "dist": [None if x == INF else x for x in snap]}
                              for v, d, upd, snap in steps],
                    "dist": [None if x == INF else x for x in dist],
                    "path5": [v + 1 for v in restore_path(parent, 4)],
                    "path4": [v + 1 for v in restore_path(parent, 3)]}
    for st in res["small"]["steps"]:
        print(st)
    print(res["small"]["dist"], res["small"]["path5"])

    # --- 2. Сравнение реализаций: разреженный и плотный графы
    rng = random.Random(14)
    res["dijkstra"] = []
    print(f"{'n':>6}{'тип':>10}{'m':>9}{'массив, с':>12}{'куча, с':>10}{'отношение':>11}")
    for n in (300, 600, 1200):
        for kind, m in (("sparse", 4 * n), ("dense", int(0.2 * n * (n - 1)))):
            adj = generate_graph(n, m, rng)
            m_real = sum(len(a) for a in adj)
            d1, _ = dijkstra_array(adj, 0)
            d2, _ = dijkstra_heap(adj, 0)
            assert d1 == d2 and INF not in d1
            t_a, t_h = timed(dijkstra_array, adj, 0), timed(dijkstra_heap, adj, 0)
            res["dijkstra"].append({"n": n, "kind": kind, "m": m_real, "t_array": t_a, "t_heap":
↪   t_h,
                                    "ecc": max(d1)})
            print(f"{n:>6}{kind:>10}{m_real:>9}{t_a:>12.4f}{t_h:>10.4f}{t_a / t_h:>11.1f}",
             ↪  flush=True)

    # --- пример восстановления пути (n = 300)
    adj = generate_graph(300, 1200, random.Random(7))
    dist, parent = dijkstra_heap(adj, 0)
    target = max(range(300), key=lambda v: dist[v])
    path = restore_path(parent, target)
    res["path_example"] = {"target": target, "dist": dist[target], "path": path}
    print("Самая удалённая вершина", target, dist[target], path)

    # --- 3. Критический путь проекта (13 работ)
    T, ES, EF, LS, LF, R, crit, order = critical_path(TASKS)
    best, best_paths = all_paths_longest(TASKS)
    assert best == T, (best, T)
    assert sorted(crit) == sorted({v for p in best_paths for v in p}), "критические работы
     ↪  расходятся с перебором"
```

```
    res["cpm"] = {"T": T, "ES": ES, "EF": EF, "LS": LS, "LF": LF, "reserve": R, "critical": crit,
                  "order": order, "paths": best_paths, "tasks": {k: [v[0], v[1]] for k, v in
                   ↪  TASKS.items()}}
    print(f"T = {T}", crit, best_paths)
    for k in sorted(TASKS):
        print(k, TASKS[k][0], ES[k], EF[k], LS[k], LF[k], R[k])

    # проверка обнаружения цикла в сетевом графике
    bad = dict(TASKS)
    bad["B"] = (7, ["A", "I"])
    try:
        topological_order(bad)
        res["cycle_check"] = "не обнаружен"
    except ValueError as e:
        res["cycle_check"] = str(e)
    print(res["cycle_check"])

    with open("res/lr10.json", "w", encoding="utf-8") as f:
        json.dump(res, f, ensure_ascii=False)
    make_plots(res)
```

6. Таблицы измерений и графики для двух реализаций Дейкстры

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Время — медиана 5 повторов; оба алгоритма запускались на одном и том же графе из вершины 0, расстояния сравнивались (`assert`) — различий нет, все вершины достижимы.

Таблица 5. Время работы двух реализаций алгоритма Дейкстры

n	Тип графа	m	Массив, с	Куча, с	Отношение
300	разреженный	1 200	0,0017	0,0002	10,4
300	плотный	17 940	0,0019	0,0008	2,4
600	разреженный	2 400	0,0074	0,0004	18,6
600	плотный	71 880	0,0091	0,0042	2,2
1 200	разреженный	4 800	0,0327	0,0010	32,2
1 200	плотный	287 760	0,0488	0,0193	2,5

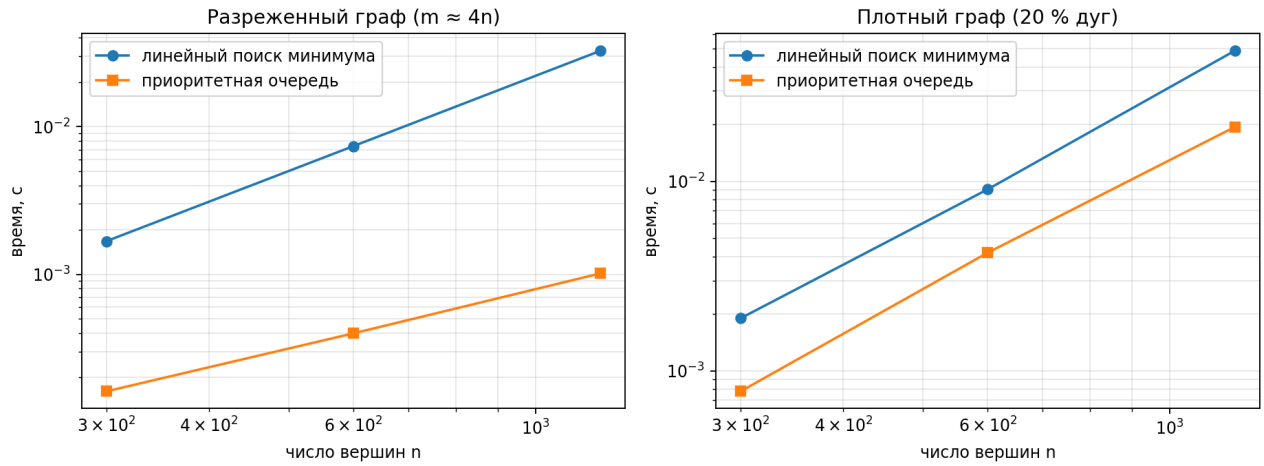


Рис. 1. Время работы Дейкстры (логарифмические шкалы)

Таблица 6. Рост времени при удвоении n

Переход	Рост n	Массив, разреж.	Куча, разреж.	Массив, плотн.	Куча, плотн.	Теория (массив)	Теория (куча, разреж.)
300 → 600	2	4,4	2,5	4,8	5,4	4	≈ 2,2
600 → 1200	2	4,4	2,5	5,4	4,6	4	≈ 2,2

На разреженных графах преимущество кучи велико и растёт с n (от 10,4 до 32,2 раза), на плотных остаётся почти постоянным (2,2–2,5 раза). При $n = 1200$ на разреженном графе массив тратит 0,0327 с, куча — 0,0010 с. Время массива на разреженных графах растёт в 4,4 раза при удвоении n (теория $\Theta(n^2)$ — 4 раза), на плотных — в 4,8–5,4 раза (там $m = 0{,}2\,n^2$ тоже растёт в 4 раза, и слагаемое m вносит заметный вклад). Время кучи на разреженных графах растёт в 2,5 раза (теория $O(m \log n)$ — около 2,2), на плотных — в 4,6–5,4 раза, поскольку при $m = \Theta(n^2)$ число дуг растёт в 4 раза.

Пример восстановления пути. В случайном графе из 300 вершин и 1200 дуг (веса 1…80) наиболее удалённая от источника вершина имеет номер 232, расстояние до неё 162, путь: 0 → 4 → 62 → 232.

7. Расчёт сроков работ проекта

Таблица 7. Расчёт сетевого графика проекта

Работа	t	ES	EF	LS	LF	Резерв R	Критическая
A	5	0	5	0	5	0	**да**
B	7	5	12	5	12	0	**да**
C	4	12	16	13	17	1	
D	6	12	18	12	18	0	**да**
E	10	16	26	17	27	1	
F	9	18	27	18	27	0	**да**
G	3	27	30	27	30	0	**да**
H	4	18	22	32	36	14	
I	6	30	36	30	36	0	**да**
J	2	36	38	36	38	0	**да**
K	3	12	15	30	33	18	
L	5	16	21	33	38	17	
M	3	38	41	38	41	0	**да**

Длительность проекта: 41 дней. Критический путь: A → B → D → F → G → I → J → M (8 работ).

Проверка: перебор всех путей сетевого графика (функция `all_paths_longest`) даёт тот же срок (41 дней) и тот же единственный наибольший путь; множество критических работ совпало.

Анализ резервов.

- Работы A, B, D, F, G, I, J, M имеют нулевой резерв — их задержка увеличивает срок проекта.
- Работы C и E имеют резерв 1 день: ветвь «проектирование базы данных — серверная часть» почти критическая (путь A–B–C–E–G–I–J–M даёт 40 дней), задержка более чем на день переведёт её в критическую.
- Работа H (подготовка документации) имеет резерв 14 дней, работа L (миграция данных) — 17 дней, работа K (развёртывание инфраструктуры) — 18 дней; их можно сдвигать в пределах резерва без изменения срока. Для K и L резерв отсчитывается от работы M: приёмка начинается на 38-й день, миграция данных должна закончиться к этому сроку.
- Срок проекта увеличился с 38 до 41 дня по сравнению с базовым набором из 10 работ из-за добавления приёмки M (3 дня) после последней работы J.

8. Дополнительное задание K7: диаграмма Ганта

Диаграмма построена по ранним срокам (ES, EF): каждая работа изображена отрезком от ES до EF; красным выделены критические работы, синим — остальные; штриховой полосой показан полный резерв времени R (от EF до LF), пунктирная линия — срок проекта.

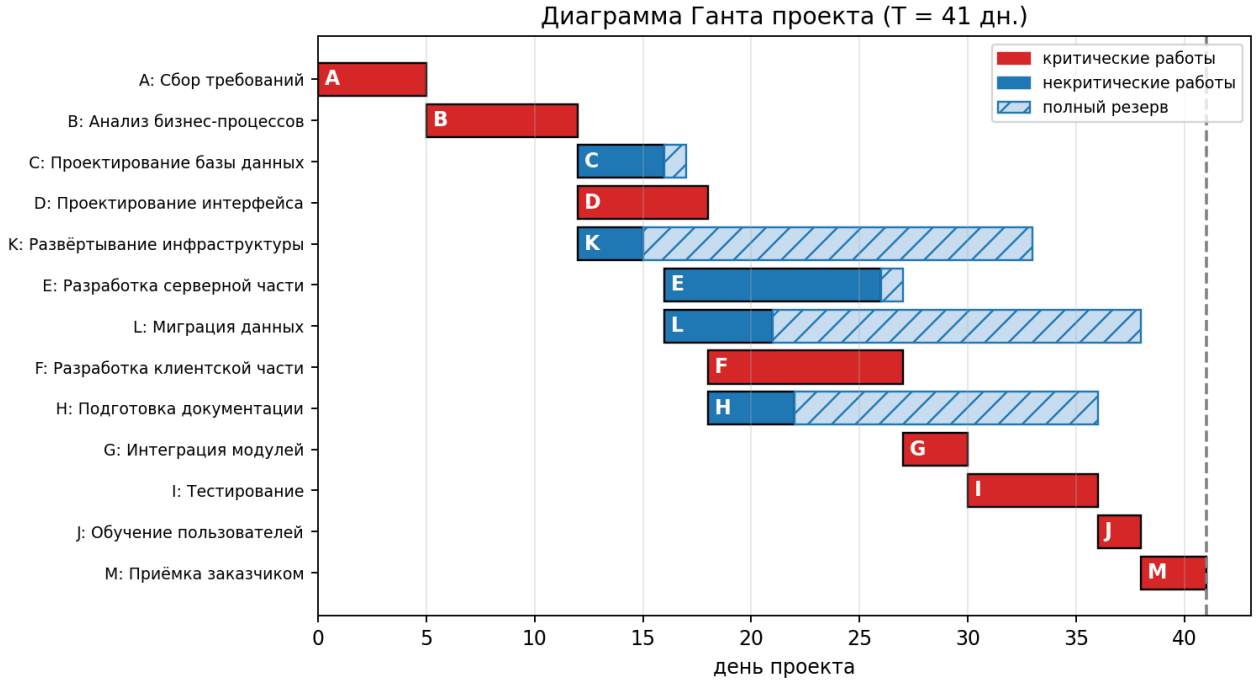


Рис. 2. Диаграмма Ганта проекта (T = 41 дн.)

По диаграмме видно, что критические работы образуют непрерывную цепочку от начала до конца проекта без зазоров, а некритические работы (C, E, H, K, L) можно сдвигать вправо в пределах штриховых полос. Параллельная ветвь C–E лишь на день короче критической ветви D–F, поэтому диаграмма наглядно показывает «почти критический» участок.

9. Выводы

1. На разреженных графах реализация Дейкстры с приоритетной очередью многократно быстрее реализации с линейным поиском минимума, и разрыв растёт с n (от 10,4 до 32,2 раза при n от 300 до 1200), что соответствует оценкам $O(m \log n)$ против $\Theta(n^2)$.

2. На плотных графах (20 % дуг) преимущество кучи сохраняется, но остаётся почти постоянным (2,2–2,5 раза): при $m = \Theta(n^2)$ обе оценки близки по порядку. Вероятная причина того, что куча остаётся быстрее, — вставки в неё происходят лишь при улучшении оценок, а не для каждой из m дуг (причина отдельно не проверялась); для проверки точки, где массив станет выгоднее, нужны эксперименты с большей плотностью (в работе не проводились).
3. Обе реализации дают одинаковые расстояния, подтверждённые алгоритмом Беллмана — Форда на 200 случайных графах; веса восстановленных путей равны расстояниям.
4. Расчёт сетевого графика из 13 работ выполняется за два прохода в топологическом порядке, $\Theta(n + m)$. Срок проекта — 41 дней; критический путь A → B → D → F → G → I → J → M состоит из 8 работ, остальные 5 работ имеют положительный резерв (1, 1, 14, 17 и 18 дней).
5. Работы вне критического пути можно сдвигать в пределах резерва без ущерба для срока; сокращение срока проекта возможно только за счёт критических работ, и только до тех пор, пока не станет критической параллельная ветвь (здесь — на 1 день, затем критическим станет путь A–B–C–E–G–I–J–M).
6. Контроль ацикличности выполняется самой топологической сортировкой: при наличии цикла (искусственно введённого в график) программа сообщает об ошибке исходных данных.

10. Ответы на контрольные вопросы

1. Сформулируйте задачу о кратчайших путях. В чём состоит свойство оптимальной подструктуры? Дан взвешенный ориентированный граф с весовой функцией w; длина пути — сумма весов его дуг; требуется найти путь минимальной длины из s в t (или из s во все вершины). Свойство оптимальной подструктуры: любой подпуть кратчайшего пути сам является кратчайшим (иначе заменой подпути на более короткий можно было бы сократить весь путь). Это даёт возможность вычислять расстояния шаг за шагом (релаксация). В тестовом графе путь $1 \to 3 \to 6 \to 5$ кратчайший, и его подпуть $1 \to 3 \to 6$ (длина 11) — кратчайший до вершины 6.

2. Опишите шаг релаксации ребра. Для дуги (v, u) с весом w: если $d[v] + w < d[u]$, то $d[u] := d[v] + w$ и $parent[u] := v$ — найден более короткий путь до u через v. В нашем расчёте релаксация дуги $3 \to 4$ улучшила оценку $d[4]$ с 22 до 20.

3. Почему алгоритм Дейкстры неприменим при отрицательных весах? Приведите контрпример. Алгоритм считает, что вершина с минимальной оценкой уже получила окончательное значение; при отрицательной дуге более поздний путь может оказаться короче. Контрпример: дуги $s \to a$ (2), $s \to b$ (3), $b \to a$ (−2), $a \to t$ (1). Дейкстра сначала просматривает a ($d = 2$) и фиксирует $d[t] = 3$; позже через b оценка a уменьшается до 1, но t уже не пересматривается. Верное расстояние до t: $s \to b \to a \to t = 3 - 2 + 1 = 2$. Эксперимент: реализация `dijkstra_array` даёт $d[t] = 3$, Беллман — Форд — 2.

4. Сравните две реализации Дейкстры. При какой плотности графа выгодна каждая? Массив: $\Theta(n^2)$, не зависит от m, проще; куча: $O(m \log n)$, выигрывает на разреженных графах. Теоретически массив выгоднее при очень плотных графах ($m \approx n^2$), поскольку у кучи добавляется множитель $\log n$ и накладные расходы. В нашем опыте (Python) куча была быстрее на обоих типах: в 10,4 до 32,2 раза на разреженных и 2,2–2,5 раза на плотных графах с 20 % дуг; выигрыш массива ожидается лишь при ещё более высокой плотности (не исследовалось).

5. Чем алгоритм Беллмана — Форда отличается от алгоритма Дейкстры? Беллман — Форд $n-1$ раз релаксирует все дуги: $\Theta(nm)$, допускает отрицательные веса и обнаруживает отрицательные циклы. Дейкстра работает быстрее ($O(m \log n)$ или $\Theta(n^2)$), но требует неотрицательных весов. В работе Беллман — Форд использован как независимый эталон проверки.

6. Что такое топологическая сортировка? Для каких графов она определена? Это такое упорядочение вершин, при котором каждая дуга ведёт от более ранней вершины к более поздней. Она определена только для ориентированных ациклических графов (DAG): при наличии цикла вершины цикла нельзя расположить в линейном порядке. Для сетевого графика проекта — порядок выполнения работ с учётом предшествования.

7. Как обнаружить цикл при топологической сортировке? В алгоритме Кана

вершины с нулевой входящей степенью последовательно удаляются; если после завершения удалены не все вершины, оставшиеся лежат на циклах или зависят от них. В программе это условие вызывает `ValueError`; проверка показала: при искусственно введённом цикле выдаётся сообщение «Сетевой график содержит цикл».

8. Запишите формулы прямого и обратного проходов метода критического пути. Прямой: $ES_i = \max_{j \in \mathrm{Pred}(i)} EF_j$, $EF_i = ES_i + t_i$ ($ES_i = 0$ без предшественников), $T = \max_i EF_i$. Обратный: $LF_i = \min_{j \in \mathrm{Succ}(i)} LS_j$, $LS_i = LF_i - t_i$ ($LF_i = T$ без последователей). Для работы G: $ES = \max(EF_E, EF_F) = \max(26, 27) = 27$, $EF = 30$; $LF = LS_I = 30$, $LS = 27$.

9. Что такое полный резерв времени работы? Каков он у работ критического пути? $R_i = LS_i - ES_i = LF_i - EF_i$ — на сколько можно задержать работу, не увеличив срок проекта. У критических работ $R = 0$ (A, B, D, F, G, I, J, M), у некритических положителен (C и E — 1 день, H — 14, L — 17, K — 18 дней).

10. Почему сокращение длительности некритической работы не сокращает срок проекта? Срок проекта определяется самым длинным путём (критическим), и некритическая работа лежит на более коротком пути, который имеет резерв. Например, сокращение H на 3 дня ничего не меняет: у неё резерв 14 дней. Более того, и сокращение критической работы даёт эффект лишь до тех пор, пока не станет критической параллельная ветвь: путь A–B–C–E–G–I–J–M длиной 40 дней ограничивает выигрыш одним днём, поэтому сокращение G на 3 дня уменьшит срок проекта не до 38, а лишь до 40 дней.